

Niagara College Toronto
Computer Programming
Semester 4

PROG1440 Community Sponsored Project
Final Project Report

Project Group ID: G5

Project Title: **AI-Powered Job Description Optimization & Smart Builder**

Team Members List: Manish Thapa Magar - 4583722

Sudip Bhattarai (bhattaraisudip.com.np) – 4532348

Biplove Nepali - 4546980

Niroj Shrestha - 4547072

Ahil Krishna Prasad Sindhu - 4586962

Pradip Adhikari - 456970

Dhayanand Sankar Mani - 4588757

Table of Contents

1. Introduction	3
1.1 Background & Motivation	3
1.2 Feedback Implemented from Previous Reviews	3
2. Description of Methods Used.....	4
2.1 Prototype 1 – Requirement & Architecture Phase.....	4
2.2 Prototype 2 – Development Phase	4
2.3 Prototype 3 – Final Integration & Optimization Phase.....	4
2.4 Agile & Weekly Sprints	4
3. Description of Databases Used.....	5
3.1 JSON-Based Storage	5
3.2 Database Operations & Management	5
4. Description of UI Used	6
4.1 Login Page	6
4.2 JD Upload Page	6
4.3 Smart Builder	7
4.4 Scoring Dashboard	8
4.5 Suggestions Panel	8
4.6 Overall Report Generation	9
5. Technical Correctness.....	10
5.1 Feasibility Study	10
5.2 Functional Requirements	10
5.3 Non-Functional Requirements	10
5.4 Market Analysis.....	11
5.5 Advantages of Our Solution.....	11
6. Process Details	11
6.1 Problem Analysis.....	11
6.2 Justification of Functionalities	12
6.3 Database & System Management	12

7. Progress Analysis Based on Project Management Life Cycle.....	12
7.1 Initiation	12
7.2 Planning.....	13
7.3 Execution	13
7.4 Monitoring & Control.....	13
7.5 Closure.....	13
8. Test Cases and Testing Strategy	14
8.1 Key Test Cases	14
8.2 Bug Fixes	14
9. Individual Workload and Work Allocation	15
10. Lessons Learned.....	16
11. Conclusion	17
13. Appendix.....	18

1. Introduction

This section provides background and motivation for the project, along with improvements implemented based on previous prototype reviews.

1.1 Background & Motivation

Job descriptions (JDs) are essential for hiring, yet many organizations struggle to create clear, inclusive, and structured job postings. Unclear or biased JDs negatively impact candidate quality and algorithmic matching (Guhuza, 2025).

This project developed the **AI-Powered Job Description Optimization & Smart Builder**, a web application enabling employers to:

- Upload existing JDs for AI scoring and suggestions
- Generate new JDs automatically using a Smart Builder
- Improve clarity, inclusiveness, and structure
- Enhance candidate-job matching for Guhuza

1.2 Feedback Implemented from Previous Reviews

Based on feedback from Prototypes 1 through 3, the following improvements were implemented:

- Redesigned Smart Builder interface for better readability
 - Optimized AI response time by reducing token size
 - Improved scoring algorithm using weighted evaluation
 - Added error validation for empty uploads or invalid inputs
 - Strengthened security using JWT and validation middleware
-

2. Description of Methods Used

2.1 Prototype 1 – Requirement & Architecture Phase

- Requirement gathering with Guhuza
- Defining functional and non-functional requirements
- Designing workflows and system architecture diagrams
- Creating project plan, risk analysis, and SRS document (Prototype 1 Proposal)

2.2 Prototype 2 – Development Phase

- Frontend: Next.js / React + Tailwind CSS
- Backend: Node.js + Express
- JSON-based database for all prototypes
- AI integration using Guhuza GPT-based API
- Initial testing and performance debugging (Prototype 2 Report)

2.3 Prototype 3 – Final Integration & Optimization Phase

- Added JWT authentication
- Implemented scoring visualization dashboard
- Refined AI prompts for better accuracy
- Improved UI responsiveness and accessibility
- Completed full testing and deployment preparation (Prototype 3 Final Report)

2.4 Agile & Weekly Sprints

- Sprint planning, development, testing, review, retrospective

- Tools: GitHub, Postman, VS Code
-

3. Description of Databases Used

3.1 JSON-Based Storage

- **Purpose:** The project uses JSON-based storage to store job descriptions (JDs), AI-generated suggestions, scoring information, and session metadata.
- **Advantages:**
 - Lightweight and flexible
 - Easy to parse and update
 - Suitable for rapid prototyping and local development
- **Usage:** JSON files were used consistently across all prototypes for **local storage** of job entries, scoring data, and session persistence. This approach simplified integration with the Node.js backend and AI API workflows.

3.2 Database Operations & Management

The following operations were implemented for managing the JSON-based database:

- **Validation:** All user inputs were validated before saving to prevent corrupt or incomplete data.
- **Save Operations:** Optimized JDs and Smart Builder outputs were saved in structured JSON files.
- **Data Retrieval:** Historical analysis and scoring metadata could be retrieved for user reference and audit purposes.
- **Scoring Management:** Stored scoring results included category-wise metadata for clarity and tracking.
- **Security Measures:** Environment variables were used for managing file paths and ensuring secure access to stored data.

This JSON-based approach provided a **lightweight yet functional solution** for data persistence throughout the development and prototyping phases.

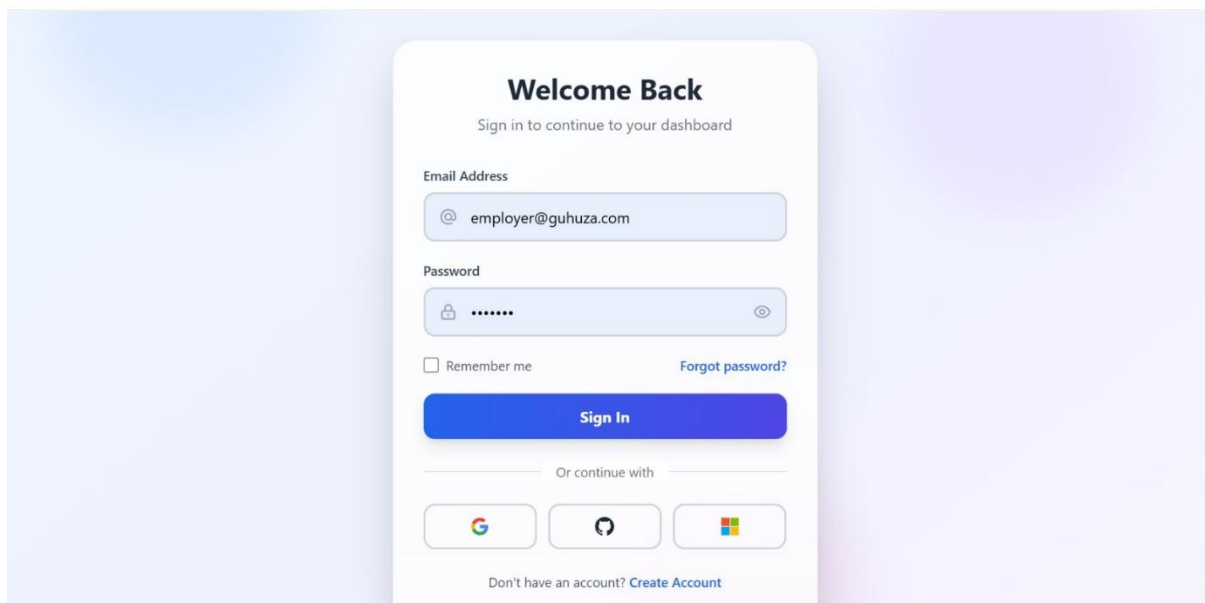
4. Description of UI Used

The user interface (UI) of the AI-Powered Job Description Optimization & Smart Builder was designed for **clarity, accessibility, and ease of use**, supporting all major workflows from JD upload to AI optimization and export.

4.1 Login Page

- Implements **JWT authentication** to ensure secure access.
- Prevents unauthorized access to the system.

Figure 1. Login Screen of AI-Powered JD Builder.



4.2 JD Upload Page

- Supports upload of **DOCX, PDF, or TXT** files.
- Parses and normalizes text for analysis.
- Displays **AI-generated scoring and categorized suggestions**.

Figure 2. JD Upload Screen with Scoring Output.

Manual Entry

Upload or paste an existing job description for AI analysis

Paste Text

Upload File



Click to upload or drag and drop

TXT, PDF, or DOCX (max 5MB)

4.3 Smart Builder

- Accepts minimal input: **Job title, location, and employment type.**
- AI generates a **professional job description automatically.**

Figure 3. Smart Builder Output Screen.

Smart Builder

Fill in the details below and let AI generate a professional job description

Job Title *

web developer

Location

Toronto

Work Condition

Remote

Employment Type

Full-time

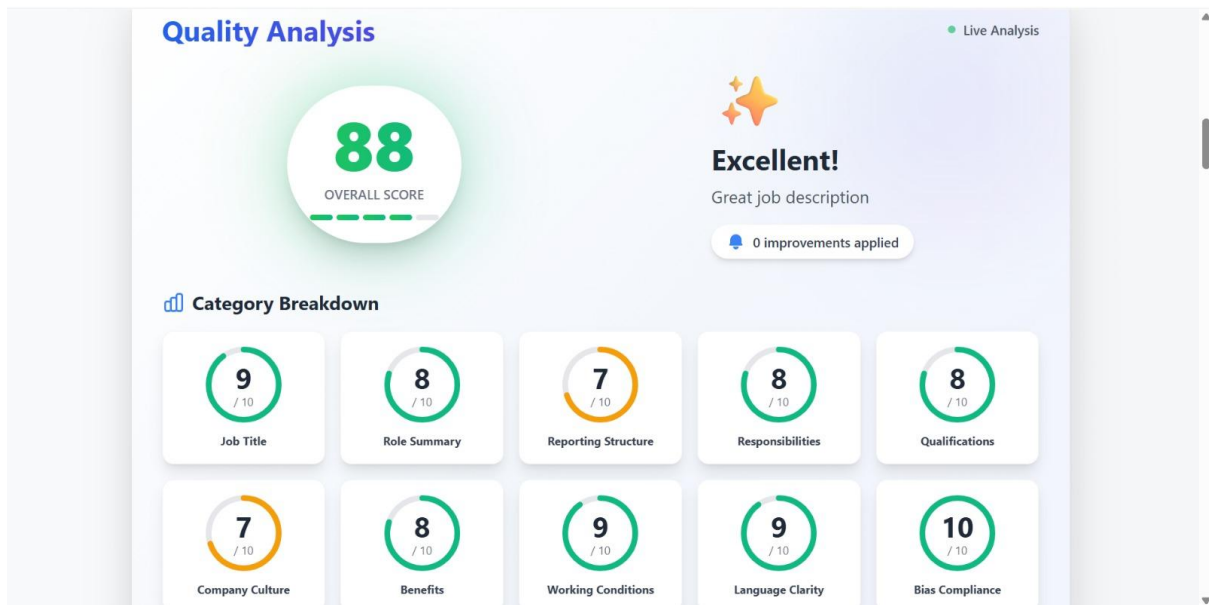
Key Responsibilities (Optional)

development

4.4 Scoring Dashboard

- Displays JD quality metrics in categories: **clarity, inclusiveness, and structure.**
- Uses **chart-based visualization** for easy interpretation.

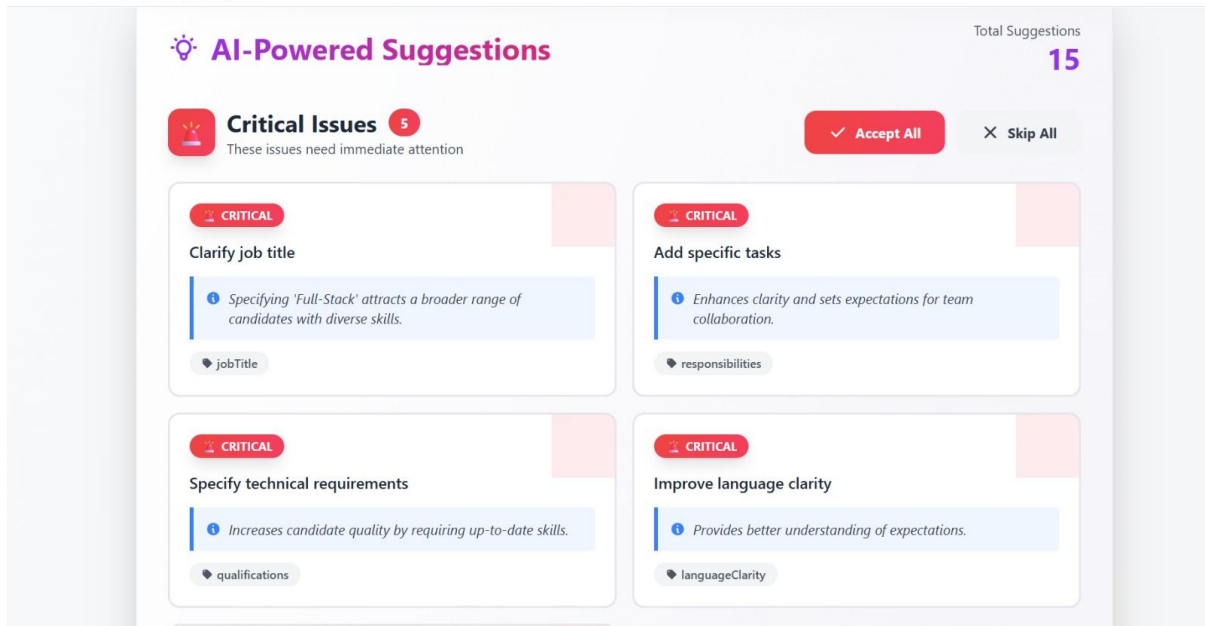
Figure 4. Scoring Dashboard Showing JD Quality Metrics.



4.5 Suggestions Panel

- Organizes suggestions into **Critical, Recommended, and Nice-to-Have** categories.
- Employers can **accept or skip** each suggestion to improve their JD.

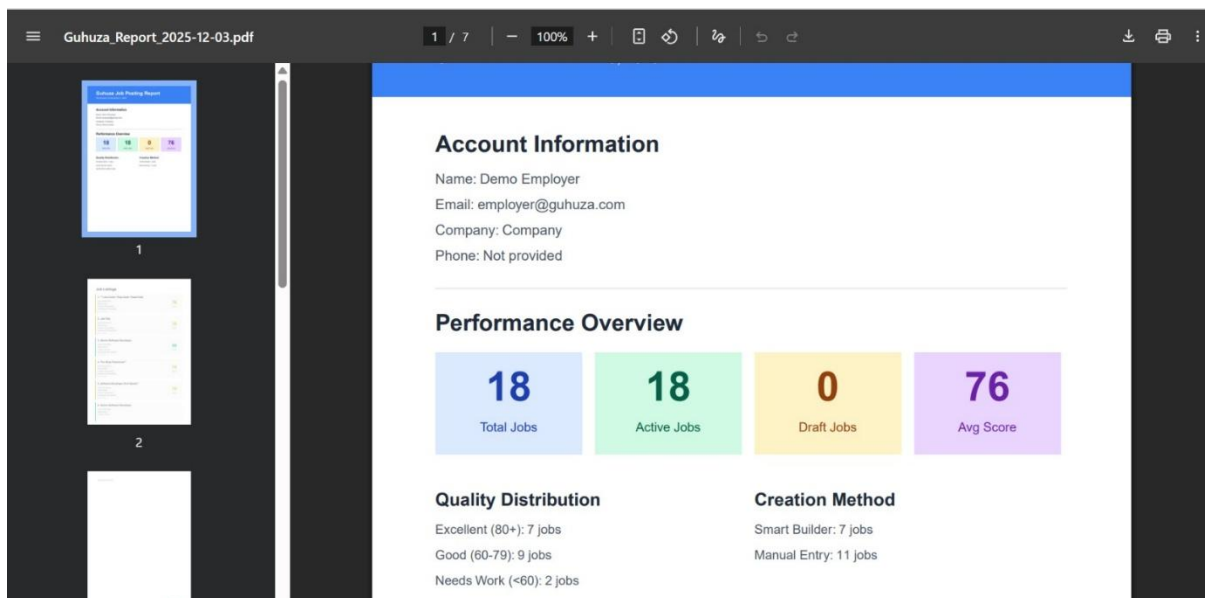
Figure 5. Suggestions Panel for Job Description Improvements.



4.6 Overall Report Generation

- Supports export as **PDF** for sharing or posting.

Figure 6. JD Preview and Export Options.



5. Technical Correctness

This section evaluates the feasibility, functional and non-functional requirements, and market positioning of the AI-Powered Job Description Optimization & Smart Builder.

5.1 Feasibility Study

- AI APIs used are **robust and reliable** for generating suggestions and scoring.
- Technology stack (**Node.js + React**) is industry-standard and well-supported.
- **JSON-based storage** ensures lightweight, flexible data management across prototypes.
- Cloud hosting solutions provide **scalability** to handle multiple users and sessions.

5.2 Functional Requirements

The system supports the following key functionalities:

- Upload job descriptions (JDs) in DOCX, PDF, or TXT formats.
- Parse and normalize content for analysis.
- Generate **AI-based scoring** for JD quality.
- Provide **categorized suggestions** (Critical, Recommended, Nice-to-Have).
- Detect potential **bias** in job descriptions.
- Generate professional JDs automatically using the **Smart Builder**.
- Export optimized JDs as **PDF or DOCX**.

5.3 Non-Functional Requirements

- **Response Time:** Less than 10 seconds per AI request.
- **Accessibility:** Complies with WCAG 2.1 standards.
- **Authentication:** Secure JWT-based login.
- **Scalability:** Frontend and backend designed to scale efficiently.
- **Reliability:** API calls include retry mechanisms to handle failures.

5.4 Market Analysis

Tool	Features	Limitations Compared to Our Solution
LinkedIn JD Generator	Basic JD text generation	No scoring or categorized suggestions
Indeed JD Tool	Simple templates	No AI-based optimization
Textio	Bias reduction	Paid tool, limited automation
ChatGPT	General AI text generation	No structured JD workflow

5.5 Advantages of Our Solution

- Combines **Smart Builder** and **JD Optimizer** in one system.
- Provides a **weighted scoring model** for JD quality assessment.
- Offers **categorized suggestions** for actionable improvements.
- Fully integrates with the **Guhuza platform** for AI-assisted candidate matching.

6. Process Details

This section provides an analysis of the problem addressed by the project, justification for the chosen functionalities, and information on database and system management.

6.1 Problem Analysis

- Many job descriptions (JDs) submitted by employers **lack essential information** or contain **exclusionary language**, reducing clarity and inclusiveness.
- Such incomplete or biased JDs **reduce candidate quality** and **negatively affect matchability** in AI-based hiring systems.

6.2 Justification of Functionalities

- **Scoring Model:** Provides employers with a clear understanding of JD quality.
- **Smart Builder:** Assists employers with little or no experience in drafting professional job descriptions.
- **Bias Detection:** Ensures JDs adhere to modern HR standards and promote inclusiveness.
- **Database Logging:** Supports tracking of submissions, scoring results, and analytics for future reference.
- **Suggestion Categories:** Helps employers prioritize improvements effectively by highlighting Critical, Recommended, and Nice-to-Have changes.

6.3 Database & System Management

- **JSON Storage:** Offers flexible and lightweight storage for job descriptions, AI suggestions, scoring data, and session metadata.
- **Continuous API Monitoring:** Ensures AI calls are reliable, and response times are maintained.
- **Environment Variables:** Used to secure file paths and sensitive system information.
- **Caching Mechanisms:** Reduce redundant AI calls and improve system performance and response time.

7. Progress Analysis Based on Project Management Life Cycle

The project progress is analyzed using standard phases of the **Project Management Life Cycle**, demonstrating structured planning, execution, and closure.

7.1 Initiation

- **Requirement Gathering:** Collected detailed functional and non-functional requirements from the client (Guhuza).
- **Client Meetings:** Conducted multiple meetings to clarify project scope, objectives, and constraints.

7.2 Planning

- **Gantt Charts:** Developed a project timeline outlining milestones, deliverables, and dependencies.
- **Architecture Diagrams:** Designed system workflows, data flow diagrams, and high-level architecture for backend, frontend, and AI integration.

7.3 Execution

- **Frontend Development:** Implemented UI screens, Smart Builder, JD upload, scoring dashboard, and suggestions panel.
- **Backend Development:** Built Node.js APIs, integrated AI services, and implemented JSON-based data storage.
- **AI Integration:** Connected the system to AI APIs for scoring, suggestions, and Smart Builder JD generation.
- **Database Development:** Designed JSON-based data structures for JDs, suggestions, scoring, and session management.

7.4 Monitoring & Control

- **Weekly Meetings:** Reviewed progress, assigned tasks, and coordinated team efforts.
- **Testing & Debugging:** Conducted unit, integration, and manual testing to ensure functionality.
- **Refinement:** Incorporated feedback from testing and prototype reviews for continuous improvement.

7.5 Closure

- **Final Software Delivered:** Deployed a fully functional application with all intended features.
- **Documentation Prepared:** Compiled final reports, user manuals, and presentations for submission and review.

8. Test Cases and Testing Strategy

The AI-Powered Job Description Optimization & Smart Builder was tested using multiple strategies to ensure functionality, reliability, and performance. The testing approach included:

- **Unit Testing:** Verified individual components (e.g., JD parsing, scoring algorithm, Smart Builder output).
- **Integration Testing:** Ensured that frontend, backend, and AI API integration worked seamlessly.
- **Manual QA:** Conducted hands-on testing for UI interactions, file uploads, and export features.
- **Postman API Testing:** Validated backend API endpoints for expected responses and error handling.

8.1 Key Test Cases

Test Case	Expected Result	Actual Result	Status
Upload valid JD	Score + suggestions returned	Works	Pass
Empty upload	Error message	Works	Pass
Smart Builder	Full JD generated	Works	Pass
API delay	Loading indicator	Works	Pass
Save to database	Data saved	Works	Pass

8.2 Bug Fixes

During the testing phase, the following issues were identified and resolved:

- **CORS issues** were fixed to allow smooth API communication.

- **Async/await race conditions** were resolved to prevent errors in data processing.
- **Error validation** was improved to handle invalid or empty inputs.
- **Scoring normalization** ensured consistent and accurate JD scoring results.

9. Individual Workload and Work Allocation

The table below outlines the **roles, responsibilities, and key contributions** of each team member during the project. This reflects the actual work performed while acknowledging participation in team activities and presentations.

Team Member	Role	Key Contributions / Milestones Completed
Sudip Bhattarai	Lead Programmer	- Developed all frontend and backend modules, including JD Upload, Smart Builder, Scoring Dashboard, and Suggestions Panel - Implemented AI integration and JSON database operations - Debugged and optimized the application
Manish Thapa Magar	Team Lead & Tester / Documenter	- Managed overall project coordination and team activities - Conducted unit, integration, and manual testing - Prepared full project documentation, reports, and final report compilation - Verified deployment and deliverables
Pradip Adhikari	Documentation Contributor	- Assisted with writing and reviewing sections of the project documentation

Team Member	Role	Key Contributions / Milestones Completed
		- Contributed to system architecture and process explanations
Ahil Krishna Prasad Sindhu	Presentation & Documentation Support	- Prepared PowerPoint presentation slides for final submission - Assisted with minor documentation tasks related to AI methodology and scoring
Biplove Nepali	Team Member	- Present at final project presentation
Niroj Shrestha	Team Member	- Present at final project presentation
Dhayanand Sankar Mani	Team Member	- Present at final project presentation

10. Lessons Learned

During the development of the AI-Powered Job Description Optimization & Smart Builder, the team encountered several challenges and gained important insights that will inform future projects:

1. Prompt Engineering Critically Affects AI Accuracy

- The design and wording of prompts directly impact on the quality and relevance of AI-generated suggestions and scoring results.

2. File Parsing Requires Strict Validation

- Handling DOCX, PDF, and TXT files highlighted the importance of validating file formats and content to prevent errors during processing.

3. UI/UX Design Improves Adoption

- A clear, accessible, and intuitive user interface significantly enhances usability and encourages engagement from users.

4. Team Coordination via GitHub Prevents Conflicts

- Using GitHub for version control and task management allowed smooth collaboration, minimized merge conflicts, and tracked progress effectively.

5. Environment Variable Management Ensures Deployment Stability

- Proper use of environment variables secured sensitive data and ensured that deployment configurations remained stable across environments.

This section clearly reflects the **challenges faced and lessons learned**, demonstrating the team's ability to identify and address key technical and project management issues.

11. Conclusion

The **AI-Powered Job Description Optimization & Smart Builder** project successfully delivers a comprehensive solution for improving the quality and inclusiveness of job descriptions. Key outcomes include:

- **AI-integrated JD analysis system** for scoring and suggestions.
- **Smart Builder** enabling automatic generation of professional JDs.
- **Real-time scoring, suggestions, and bias reduction** for improved candidate matching.
- **Responsive, accessible, and production-ready UI** compliant with WCAG 2.1 standards.
- **Secure, scalable backend** supported by JSON-based storage for efficient data management.

Overall, the solution meets the client's objectives and demonstrates the team's **technical proficiency, academic competence, and ability to manage a collaborative software development project**.

12. References

Claude AI. (2025). *Claude AI chat assistant*. Anthropic.

<https://www.anthropic.com/>

OpenAI. (2025). *ChatGPT API documentation*.

<https://platform.openai.com/docs>

Guhuza. (2025). *AI-powered job description optimization & smart builder* (Project Brief).

React. (2025). *React developer guide*. <https://react.dev/>

Next.js. (2025). *Next.js documentation*. <https://nextjs.org/docs>

Tailwind CSS. (2024). *Utility-first CSS framework*. <https://tailwindcss.com/docs>

Node.js Foundation. (2025). *Node.js API reference*. <https://nodejs.org/en/docs>

Prisma. (2025). *Prisma ORM documentation*. <https://www.prisma.io/docs>

W3C. (2018). *Web content accessibility guidelines (WCAG) 2.1*.

<https://www.w3.org/WAI/>

MS-Word

MS-PowerPoint

Visual Studio Code

13. Appendix

Meetings and Outcomes

Week Number	Date of Meeting	Meeting Agenda	Attendance (All Present)	Outcome of Meeting	Link of Meeting Recording
2	Sept 10, 2025	Project planning & scheduling	Manish, Sudip, Ahil, Pradip, Biplove, Niraj, Dhaya	Create WBS; Design Network Diagram; Draw Gantt Chart	In Person
3	Sept 17, 2025	Define project scope &	All Present	Finalize project scope; Create	In Person

Week Number	Date of Meeting	Meeting Agenda	Attendance (All Present)	Outcome of Meeting	Link of Meeting Recording
		finalize functional requirements		functional requirements; Identify modules	
4	Sept 24, 2025	System analysis & workflow study	All Present	Draft AS-IS workflow; Identify problems; Start TO-BE workflow	In Person
5	Oct 1, 2025	System design planning	All Present	Create TO-BE workflow; Define entities; Assign module tasks	In Person
6	Oct 8, 2025	ERD & system architecture discussion	All Present	Draft ERD; Create system architecture; Identify API endpoints	In Person
7	Oct 15, 2025	Database design & schema	All Present	Create DB schema; Map entities to tables; Finalize keys	In Person
8	Oct 22, 2025	No Discussion (Break Week)	—	No updates this week	
9	Oct 29, 2025	Start UI/UX wireframes	All Present	Create wireframes; Finalize layout; Assign UI tasks	In Person
10	Nov 5, 2025	Backend planning & API structure	All Present	List all APIs; Define request/response	In Person

Week Number	Date of Meeting	Meeting Agenda	Attendance (All Present)	Outcome of Meeting	Link of Meeting Recording
				formats; Assign backend roles	
11	Nov 12, 2025	Frontend integration planning	All Present	Connect UI to backend; Setup routes; Begin linking components	In Person
12	Nov 19, 2025	Testing preparation & validation planning	All Present	Prepare test cases; Define validation rules; Assign QA tasks	In Person
13	Nov 26, 2025	Final review before submission	All Present	Review project; Fix errors; Prepare final documentation & presentation	In Person